

딥러닝 프레임워크

활용 기초

과제(1) MNIST 손글씨 분류기 구현

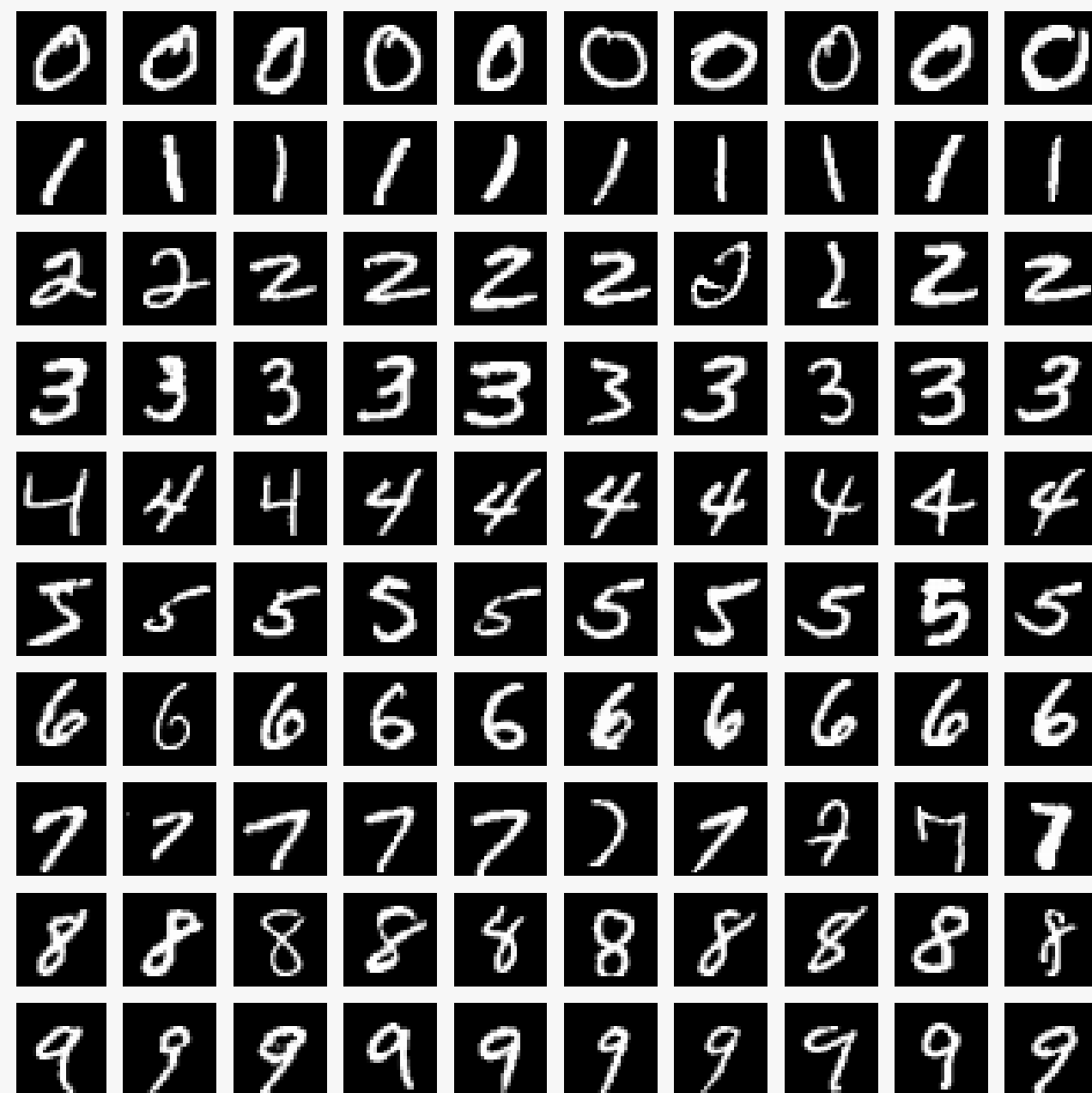
지도교수 | 정화민 교수님

제출일 | 2026.09.15

A74021 인공지능학과 김가현



MNIST 소개



01 TensorFlow

- 구글에서 공개한 오픈소스 프레임워크
- 이미지 분류, 음성 인식에 주로 쓰임

02 MNIST

- 손글씨 숫자 이미지 분류를 위한 대표적인 벤치마크 데이터셋
- TensorFlow를 통해 간편하게 불러올 수 있음
- 각 이미지는 28X28 사이즈로, 0~9 에 대한 숫자값으로 존재
- 총 60,000개의 훈련 데이터가 존재
- 10,000개의 테스트 이미지도 함께 제공

MNIST 데이터 불러오기

```
● ● ●  
  
# 라이브러리 불러오기  
from tensorflow import keras  
  
# MNIST 데이터셋 불러오기  
mnist = keras.datasets.mnist  
  
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

MNIST 데이터 불러오기



```
# MNIST 데이터셋 사이즈 확인
# train 데이터는 60000장, test 데이터는 10000장
# 한 tensor 당 28X28 사이즈
# 각각 0~9 를 표현한 손글씨 이미지
print('train_image.shape : ', train_images.shape) # 60000 X 28 X 28
print('test_image.shape : ', test_images.shape) # 10000 X 28 X 28
print('train_labels.shape : ', train_labels.shape) # 60000
print('test_labels.shape : ', test_labels.shape) # 10000
```

MNIST 데이터 시각화



```
# train 데이터 중 첫번째 이미지 확인해보기
num = train_images[0] # 28 X 28

for i in range(28) : # 세로길이 28픽셀
    for j in range(28) : # 가로길이 28픽셀
        print('{:4d}'.format(num[i][j]), end='')
        # num[0][0] num[0][1] num[0][2] ... num[0][27] num[1][0] num[1][1] ... num[27][27]
        # 각 픽셀값 출력 (0 ~ 255 사이)
    print() # 줄바꿈
```

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	3	18	18	18	126	136	175	26	166	255	247	127	0	0	0
0	0	0	0	0	0	0	0	30	36	94	154	170	253	253	253	253	253	225	172	253	242	195	64	0	0	0
0	0	0	0	0	0	0	0	49	238	253	253	253	253	253	253	253	251	93	82	82	56	39	0	0	0	0
0	0	0	0	0	0	0	0	18	219	253	253	253	253	253	198	182	247	241	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	80	156	107	253	253	205	11	0	43	154	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	14	1	154	253	90	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	139	253	190	2	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	11	190	253	70	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	35	241	225	160	108	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	81	240	253	253	119	25	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	45	186	253	253	150	27	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	16	93	252	253	187	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	249	253	249	64	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	46	130	183	253	253	207	2	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	39	148	229	253	253	253	250	182	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	24	114	221	253	253	253	253	201	78	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	23	66	213	253	253	253	253	198	81	2	0	0	0	0	0	0	0	0
0	0	0	0	0	0	18	171	219	253	253	253															

MNIST 데이터 시각화



```
# train 데이터 중 첫번째 이미지의 정답 (label) 확인하기  
print('train_labels[0] : ', train_labels[0]) # 5
```

MNIST 이미지 전처리

```
# 이미지 데이터 전처리
# 정규화 : 0~255 사이의 값들을 0~1 사이로 조정
train_images_norm = train_images / 255.0
test_images_norm = test_images / 255.0

num = train_images_norm[0] # 28 X 28

for i in range(28) : # 세로길이 28픽셀
    for j in range(28) : # 가로길이 28픽셀
        print('{:5.2f}'.format(num[i][j]), end='')
        # num[0][0] num[0][1] num[0][2] ... num[0][27] num[1][0] num[1][1] ... num[27][27]
        # 각 픽셀값 출력 (0 ~ 1 사이)
    print() # 줄바꿈
```

MNIST 이미지 전처리

0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01	0.07	0.07	0.07	0.49	0.53	0.69	0.10	0.65	1.00	0.97	0.50	0.00	0.00	0.00
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.12	0.14	0.37	0.60	0.67	0.99	0.99	0.99	0.99	0.99	0.88	0.67	0.99	0.95	0.76	0.25	0.00	0.00	0.00
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.19	0.93	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.98	0.36	0.32	0.32	0.22	0.15	0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.07	0.86	0.99	0.99	0.99	0.99	0.99	0.99	0.78	0.71	0.97	0.95	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.31	0.61	0.42	0.99	0.99	0.80	0.04	0.00	0.17	0.60	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.60	0.99	0.35	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.55	0.99	0.75	0.01	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.04	0.75	0.99	0.27	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.14	0.95	0.88	0.63	0.42	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.32	0.94	0.99	0.99	0.47	0.10	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.18	0.73	0.99	0.99	0.59	0.11	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.06	0.36	0.99	0.99	0.73	0.0						

MNIST 모델 구현

```
# 모델 build (모델의 구조를 정의)
# keras.Sequential : layer를 순차적으로 쌓겠다.
model = keras.Sequential([
    keras.Input(shape=(28, 28)),
    keras.layers.Flatten(), # Flatten : tensor를 vector화 시키겠다. (입력이 28X28 -> 784)
    keras.layers.Dense(10, activation='softmax') # Dense : 이전 layer와 연결하겠다.
                                                    # 0~9 에 해당하는 출력이 10개 -> softmax 확률표현으로
])
```

왜 Softmax Regression인가

10개의 숫자(0~9)를 분류하는 다중 클래스 분류 문제이기 때문

Dense Layer에서 각 클래스에 대한 점수(logit)를 계산

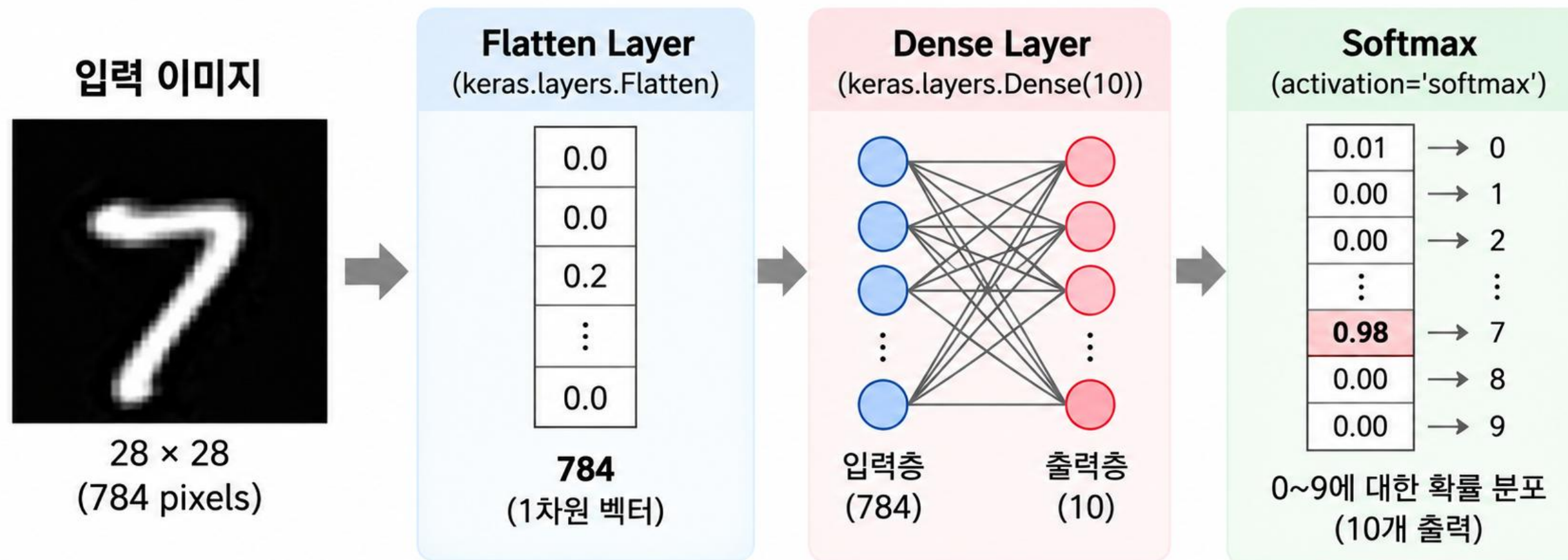


Softmax를 통해 10개 클래스에 대한 확률값으로 변환



가장 높은 확률을 가진 클래스를 최종 예측값으로 선택

MNIST 모델 구현



MNIST 모델 구현

```
# 모델 compile (모델을 어떤 규칙으로 학습하고 평가할지 정함)
model.compile(
    optimizer='adam', # optimizer는 'adam' 을 쓸 것 (loss가 감소하도록 가중치를 update하겠다)
    loss='sparse_categorical_crossentropy', # 손실함수 계산은 cross entropy 방식으로 계산할 것
                                           # 예측값과 실제 label 사이의 오차 계산
    metrics=['accuracy'] # 이 모델의 성능은 정확도로 측정할 것
)
```

MNIST 모델 학습

```
# 모델 train (fit)
model.fit(train_images_norm, train_labels, epochs=1, batch_size=32, validation_split=0.1)
# 정규화된 train 데이터와 label 60000개를 이용하여 학습
# epoch = 1 (54000장에 대해 1회만 학습하겠다)
# batch_size = 32 (54000장을 32개씩 넣겠다)
# validation_split = 0.1 (6000장은 validation 용으로 사용하겠다. 실제 학습은 54000장으로 진행)
```

MNIST 결과 확인 및 모델 평가



```
# 모델 evaluation
model.evaluate(test_images_norm, test_labels)
# test 데이터와 레이블 (문제와 답) 을 주고 얼마나 맞추는지 평가
# [0.31276148557662964, 0.9143000245094299] # 10000개의 test 문제 중 약 91.43% 를 맞췄음
# 대략 9143/10000장
```

MNIST 결과 확인 및 모델 평가

**정규화된 MNIST 데이터를 이용하여 Softmax Regression 모델을
1 Epoch 학습한 결과, 테스트 데이터에서 91.43%의 정확도를 확인하였다.**

- 실험 환경 : Google Colab, TensorFlow/Keras 기반 Python 환경
- 학습 조건 : Train Data 54,000개, Validation Data 6,000개, Test Data 10,000개, 1 Epoch, Batch size 32
- 평가 결과 : Test Loss 0.3128, Test Accuracy 0.9143
- 단일 Dense Layer를 사용하는 Softmax Regression 모델로도 약 91%의 분류 정확도를 확인하였다.
이후 Epoch와 이미지 정규화 여부에 따른 성능 차이를 추가로 비교한다.

Epoch 변화에 따른 모델 성능 비교

Epoch	Test Loss	Test Accuracy	비고
1	0.3128	91.43%	기준 모델 (Baseline)
5	0.2687	92.50%	정확도 상승 / Loss 감소
20	0.2694	92.79%	정확도 개선폭 감소 / Loss 소폭 증가

동일 조건 : 정규화 O / Batch size 32 / Validation Split 0.1 / 동일한 Softmax Regression 구조

Epoch 변화에 따른 모델 성능 분석

원인

Epoch가 증가하면서 동일한 학습 데이터를 반복적으로 학습하여 모델의 가중치가 추가로 조정되었다. 다만 현재 모델은 Flatten + Dense(10)으로 구성된 단순한 Softmax Regression 모델이므로, 복잡한 손글씨 이미지의 특징을 표현하는 데 한계가 있다.



결과

Epoch 1 → 5에서는 정확도가 91.43% → 92.50%로 상승하고 Loss가 감소하였다.
반면 Epoch 5 → 20에서는 정확도가 0.29%p 증가하는 데 그쳤으며, Loss 는 0.2687 → 0.2694로 소폭 증가하였다.

Epoch 변화에 따른 모델 성능 분석

**학습 횟수를 증가시키는 것만으로는
일정 수준 이후 성능 개선에 한계가 있음을 확인하였다.**

따라서 단순히 Epoch를 증가시키기보다, Hidden Layer 추가 등 모델 구조를 확장하는 방식을 고려할 수 있다.

Validation Split 비율에 따른 모델 성능 비교

Validation Split	Test Loss	Test Accuracy	비고
0.1	0.2687	92.50%	기준 모델 (Baseline)
0.2	0.2716	92.39%	정확도 소폭 감소 / Loss 소폭 증가 (유의미한 차이 없음)
0.3	0.2743	92.53%	정확도 소폭 증가 / Loss 소폭 증가 (유의미한 차이 없음)

동일 조건 : 정규화 0 / Batch size 32 / 5 Epoch / 동일한 Softmax Regression 구조

Validation Split 비율에 따른 모델 성능 분석

**Validation Split 비율을 0.1에서 0.3까지 변경하였으나,
정확도는 92%대로 유사하게 나타나 뚜렷한 성능 차이를 확인하기 어려웠다.**

단일 실행 결과이므로 작은 성능 차이에 대한 해석에는 주의가 필요하다.

정규화 여부에 따른 모델 성능 비교

정규화 여부	Test Loss	Test Accuracy	비고
O	0.2687	92.50%	기준 모델 (Baseline)
X	5.9680	89.23%	정확도 감소 / Loss 크게 증가

동일 조건 : Batch size 32 / Validation Split 0.1 / 5 Epoch / 동일한 Softmax Regression 구조

정규화 여부에 따른 모델 성능 분석

**정규화를 적용한 경우, 적용하지 않은 경우보다
더 높은 정확도와 낮은 Loss를 보여 모델 성능이 개선되었다.**

정규화를 적용하지 않은 경우 입력값의 범위가 커지면서 학습이 불안정해질 수 있으며,
본 실험에서는 정확도가 3.27%p 감소하고 Loss가 크게 증가하였다.

최종 결론

동일한 모델 구조에서도 학습 조건과 데이터 전처리에 따라 서로 다른 성능 변화가 나타남을 확인하였다.

- Epoch 증가에 따라 초기에는 성능이 개선되었으나, 일정 수준 이후에는 성능 개선폭이 감소하였다.
- Validation Split 을 0.1~0.3으로 변경한 결과, 정확도는 92%대로 유사하여 뚜렷한 성능 차이를 확인하기 어려웠다.
- 정규화를 적용한 경우
정확도가 증가하고 Loss가 크게 감소하여, 입력 데이터의 스케일 조정이 모델 성능에 영향을 미침을 확인하였다.

한계점 및 의의

한계점

- 동일한 조건으로 모델을 반복 학습했을 때 실행마다 Loss 와 Accuracy에 일부 차이가 발생하였다.
- 모델의 초기 가중치가 무작위로 설정되는 등 학습 과정에 확률적 요소가 있으나, 본 실험에서는 Random Seed를 별도로 고정하지 않았다. 따라서 작은 성능 차이를 절대적인 차이로 해석하는 데 한계가 있다.

의의

- 동일한 Softmax Regression 구조를 유지한 상태에서 Epoch, Validation Split, 정규화 여부를 각각 변경하여 학습 조건에 따른 성능 변화를 비교하였다.

단순한 학습 횟수 증가뿐 아니라 모델 구조 확장을 통한 성능 개선 가능성을 확인하였다.

참고 논문

Deep, Big, Simple Neural Nets for Handwritten Digit Recognition

Cireşan, D. C., Meier, U., Gambardella, L. M., & Schmidhuber, J. (2010). Neural Computation, 22, 3207–3220.

- 연구 대상 : MNIST 손글씨 숫자 분류
- 실험 요인 : Hidden Layer 수 및 Layer별 뉴런 수
- 결과 : Hidden Layer와 뉴런 수를 확장하고 학습 이미지를 변형해 데이터를 늘린 MLP에서 높은 분류 성능을 확인
- 본 실험에서 확인한 Softmax Regression의 성능 한계에 대해, Hidden Layer 및 뉴런 수 확장을 통한 모델 구조 개선 가능성을 선행연구를 통해 확인하였다.